



Product Integration (PI)

PI Overview

Required PA Information

Intent

Integrates and delivers the solution that addresses functionality, performance, and quality requirements.

Value

Increases customers' satisfaction by giving them a solution that meets or exceeds their functionality and quality requirements.

Additional Required PA Information

If Safety is included in the selected view: ensure safety requirements are addressed within the relevant integration strategies, processes, documentation, and activities.

If Security is included in the selected view: ensure security requirements are addressed within the relevant integration strategies, processes, documentation, and activities.

Explanatory PA Information

Practice Summary

Level 1

PI 1.1 Assemble solutions and deliver to the customer.

Level 2

PI 2.1 Develop, keep updated, and follow an integration strategy.

PI 2.2 Develop, keep updated, and use the integration environment.

PI 2.3 Develop, keep updated, and follow procedures and criteria for integrating solutions and components.

PI 2.4 Confirm, prior to integration, that each component has been properly identified and operates according to its requirements and design.

PI 2.5 Evaluate integrated components to ensure conformance to the solution's requirements and design.

PI 2.6 Integrate solutions and components according to the integration strategy.

Level 3

PI 3.1 Review and keep updated interface or connection descriptions for coverage, completeness, and consistency throughout the solution's life.

PI 3.2 Confirm, prior to integration, that component interfaces or connections comply with interface or connection descriptions.

PI 3.3 Evaluate integrated components for interface or connection compatibility.

Additional PA Explanatory Information

Product integration activities include:

- Using recorded integration strategies and procedures
- Using a single build or an iterative build of solution components
- Verifying and validating each build

Note that in these practices, the terms solution and solution component include products, services, service systems, and their components.

Preparing for integration is part of early planning and work activities. It involves developing and recording the:

- Integration strategy
- Integration environment
- Integration procedures and criteria

When integrating products, it is critical to manage and ensure compatibility among the interfaces or connections of product components. Component interfaces or connections can be both internal and external.

Verify and validate each successive build according to the integration strategy, in the target environment, and according to procedures and criteria.

Use automated builds and continuous integration for completed product components to achieve highest quality. Some products can be integrated using automated builds and continuous integration of the completed product components. The last integration phase may occur when they are deployed at the intended operational environment.

Related Practice Areas

Technical Solution (TS)

Context Specific

Agile Development

Context Tag:	Agile Development
Context:	Integrate agile techniques and ceremonies with other processes.

Agile teams typically employ automation and may use DevSecOps processes for unit testing, regression testing, system testing, and continuous builds, to reduce human effort as much as possible. These techniques increase productivity and help to detect defects early in the product development lifecycle.

Data

Context Tag:	CMMI-DATA
Context:	Use processes to incorporate data management best practices as an integral part of the solution.

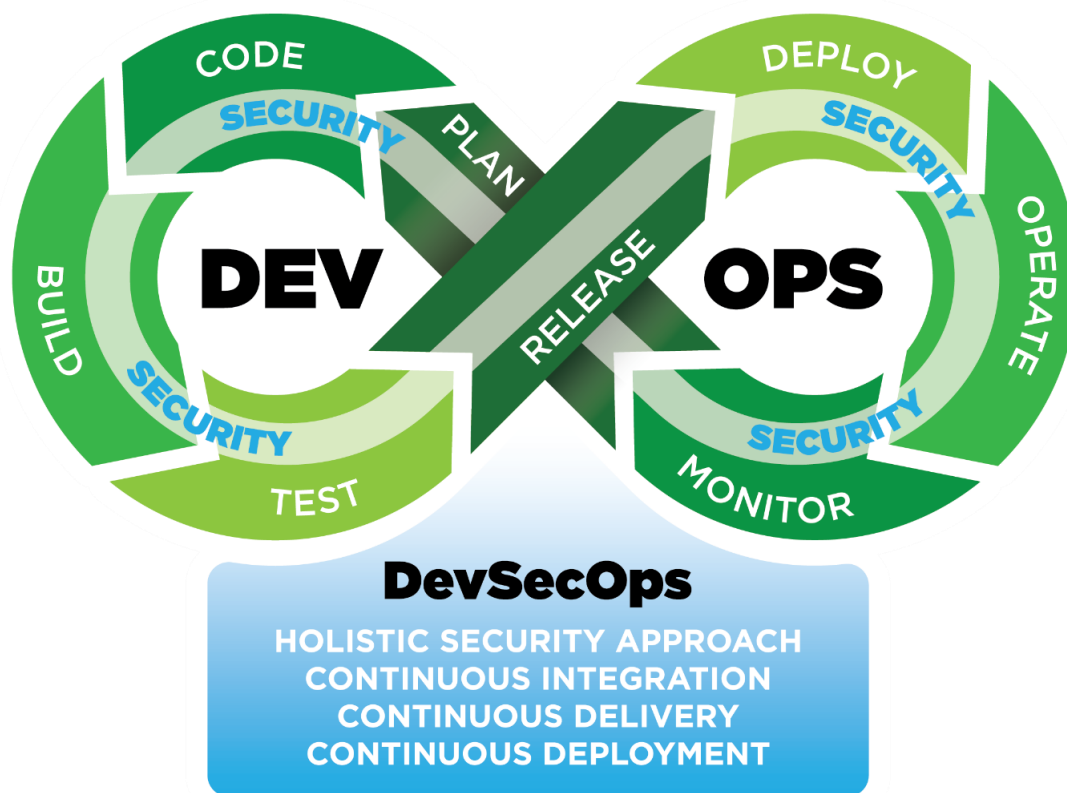
Incorporate data business rules into the product integration strategy, environment procedures, criteria, and activities. For example, consider data aggregation, data cleansing, source data optimization and prevention of data duplication, data conversions, data uploads to systems and platforms, interfaces or connections, exception handling, data transport, data storage, data archival or retirement, data sovereignty, and protections for data security and data privacy.

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

DevSecOps teams typically employ automation to develop a build and deployment pipeline that begins with Continuous Integration (CI) when a developer checks in code to source control, after which automated builds and unit tests are run. Once integration is complete, automated Continuous Delivery (CD) processes release the code into the test environment where automated regression and system testing is performed to prepare the code for deployment. Finally, CD refers to the ability to automatically deploy a developer's changes from the repository to production. It addresses the historic problem of manually transferring code to operations teams that rely on and use mostly manual processes that slow delivery to the customer. It builds on the benefits of CD by automating the last stage in the pipeline as illustrated here. Refer to [Figure PI-1: DevSecOps Approach](#).

Figure PI-1: DevSecOps Approach



Services

Context Tag:	CMMI-SVC
Context:	Use processes to deliver, manage, and improve services to meet customer needs.

When service systems are complex and comprised of multiple components, e.g., a combination of system components and services, the organization may need to sequence or integrate the services to provide a single customer facing service. In this context, the Product Integration practices provide an approach to managing and integrating multiple system and service components or service providers. Applying Product Integration practices to processes enables an organization to seamlessly integrate interdependent services from various internal and external service providers into end-to-end services to meet business requirements.

Level 1

PI 1.1

Required Practice Information

Practice Statement

Assemble solutions and deliver to the customer.

Value

Enables customer satisfaction by delivering a usable solution.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Requirements for assembly and delivery determine how the product will be delivered to the customer. For example, requirements will be different if the product is delivered as a download, as a package shipped to the customer, or delivered and installed at an operational site.

Example Activities

Example Activities	Further Explanation
Assemble solution.	
Record all necessary information to install and use the solution.	Necessary information may include: • Configuration • Solution component types and serial numbers • Physical and functional layout • Installation and tracking information • Contact information
Use applicable methods to package and deliver the solution.	Documentation packaging and delivery methods may include: • Hardcopy documents • CDs or DVDs • Online help • Cloud-based repository • Ebooks • Mobile applications • Website links for downloads
Deliver the solution and related documentation; confirm receipt.	

Example Work Products

Example Work Products	Further Explanation
Assembled solutions and related documentation	

Level 2

PI 2.1

Required Practice Information

Practice Statement

Develop, keep updated, and follow an integration strategy.

Value

Ensures that the product will meet customer requirements given available resources.

Additional Required Information

The product integration strategy includes:

- Integration and evaluation of solutions and components, e.g., as a single build or a series of builds
- Management of interfaces or connections
- Establishment of the integration environment
- Recorded evaluation results

Explanatory Practice Information

Additional Explanatory Information

Record the product integration strategy. When developing the integration strategy, follow the technical approach developed during planning, addressing the solutions chosen and designs developed during the design effort.

Developing an integration strategy can involve identifying and evaluating several alternative integration strategies or sequences.

Review the strategy with affected stakeholders to promote commitment and understanding.

Example Activities

Example Activities	Further Explanation
Identify and record product components to be integrated.	
Identify how solutions and components will be verified and validated during integration.	This includes verifications and validations to be performed on interfaces or connections.
Identify alternative integration strategies.	Possible strategies include: • Big bang • Incremental • Top-down • Bottom-up

Example Activities	Further Explanation
Select the best integration strategy.	Align the integration strategy to availability of: • Product components • The integration environment • Test tools and equipment • Procedures and criteria • Affected stakeholders • People with the appropriate skills
Periodically review the product integration strategy and revise it as needed.	Ensure that changes in production and delivery schedules have not adversely impacted the integration sequence.
Record and communicate decision rationale and status.	

Example Work Products

Example Work Products	Further Explanation
Product integration strategy	Typically includes how the: • Solutions and components will be integrated and evaluated • Interfaces or connections will be managed • Integration environment will be established • Evaluation results will be recorded The product integration strategy may also include: • The sequence in which components will be available • Whether components will be integrated and evaluated as a single build or a series of builds • The frequency of the builds, e.g., continuous using a tool, nightly, or event-driven • Features to be included and evaluated in each build when using a series of builds • How models, prototypes, and simulations will be used to assist in evaluating a component, including its interfaces or connections • How procedures and criteria will be defined • How test tools and equipment will be made available • How the solution hierarchy will be managed • How evaluation exceptions will be handled
Recorded rationale for selecting or rejecting alternative product integration strategies	
Build pipeline	A set of automated processes that allow developers to compile, build, and deploy code to the target environment throughout the lifecycle.

Example Work Products	Further Explanation
Selected integration strategy	

Related Practice Areas

Decision Analysis and Resolution (DAR)

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

DevSecOps teams incrementally build an automated Continuous Integration / Continuous Delivery (CI/CD) pipeline that spans initial build through automated releases. The integration strategy is implemented through the scripts and tool chain, however there is an integration architecture and pipeline set up that is defined before DevSecOps teams start configuring the tools. The team needs to decide between multiple architectural options which often requires formal decision-making. The goal is to reduce the risk of encountering integration issues that occur when teams wait for the end of a milestone, initiative, or a Sprint to merge all the developers' code as soon as possible during development. The code is integrated, and any integration is tested by automated test cases or sequences to look for an error. A primary benefit of CI is that it saves time and cost during the development cycle by identifying and addressing issues early. CI will also provide an opportunity to expose how well the built-in security measures have been working or if more action needs to be taken. It also reduces the time spent on bug fixes and regression by providing continuous feedback to the development team, allowing them to take corrective action when the bugs are first identified.

Security

Context Tag:	CMMI-SEC
Context:	Use processes to incorporate security considerations as an integral part of the solution, work, project, and organization.

Ensure the integration strategy incorporates appropriate security controls and consideration of third-party components. Ensure the strategy addresses:

- Level of automation
- Tool use
- Specification scope
- Disabling of services and removal of software that is not needed
- Consideration of threat scenarios
- Usage of adapted tooling to scrutinize moving perimeters and dynamics on a network
- Removal of unnecessary logins

- Changing of default passwords and logins at specified periods, e.g., release, deployment, installation, decommissioning

Consider the following configuration hardening challenges when addressing the integration strategy:

- Identifying hardening settings based on threat scenarios and intended functionality. In solutions, intended functionality determines the services, ports, and interfaces or connections that are redundant or unnecessary and thus how hardening may be achieved.
- Multiple configuration settings and details
- Heterogeneous, complex, or user-hostile interfaces or connections and vulnerabilities that could have been introduced through design or automation

PI 2.2

Required Practice Information

Practice Statement

Develop, keep updated, and use the integration environment.

Value

Provides an effective risk mitigation technique to ensure that the solution and components are integrated correctly.

Additional Required Information

Verify and validate the integration environment before use.

Explanatory Practice Information

Additional Explanatory Information

The integration environment can either be acquired or developed. When developing the integration environment, consider reusing existing organizational resources and perform build, buy, or reuse analyses. Environment requirements can include equipment, software, or other resources.

The environment required at each step of the integration process can include test equipment, simulators (taking the place of unavailable product components), product components, and recording devices.

The product integration environment can be a major development for first-time or complex projects. For small development efforts, the integration environment may be as simple as a directory structure.

Example Activities

Example Activities	Further Explanation
Develop requirements for the integration environment.	
Develop verification and validation procedures and criteria for the integration environment.	This is for validating the integration environment and not for integrating the product. Verify and validate the integration environment to ensure it satisfies the requirements in accordance with the integration strategy.
Decide whether to build, buy, or reuse the integration environment.	This can be the whole environment or just parts of it.
Develop or acquire an integration environment.	Examples of activities in developing or acquiring an integration environment include: • Planning • Requirements development • Technical solutions • Verification • Validation • Risk management
Verify and validate the integration environment.	
Use the integration environment.	The integration environment may be used for testing and other preparation work prior to solution and component integration.
Revise the integration environment as needed.	Dispose of integration environment parts that are no longer useful.
Communicate with affected stakeholders.	

Example Work Products

Example Work Products	Further Explanation
Verified and validated environment for product integration	
Build, buy, or reuse analyses	
Support documentation for the integration environment	

Related Practice Areas

Decision Analysis and Resolution (DAR)

Requirements Development and Management (RDM)

Verification and Validation (VV)

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

A Continuous Integration (CI) environment consists of four key elements:

- **CI Tool Solution:** Development teams use CI tools to automate parts of the build and to develop a recorded development process. When selecting a CI software solution, consider:
 - On-premises or hosted
 - Supports containerization
 - Ease of use
 - Security
- **Tools:** In DevSecOps, a continuous and automated delivery cycle is critical to making fast and reliable delivery possible. This requires proper Continuous Integration / Continuous Delivery (CI/CD) tools. A CI/CD tool solution needs to leverage a team's workflow to best leverage the automation feature and develop a reliable CI/CD pipeline. It also needs to integrate with continuous testing software.
- **Version Control System:** The version control system, or source control repository, records and manages changes to files and folders. It records the history of changes and enables retrieval of previous versions, if needed. The version control system allows different developers to work on their own branch, resolve editing conflicts, and then merge the code.
- **Hosting Environment:** Virtual Machine (VM), Container, and Version Control System
 - **Virtual Machine:** A VM runs a guest operating system on a host computer. VMs behave like a single computer with its own disk(s), memory, and video card, and behave as if it has its own dedicated resources. Having multiple operating systems enables each server to run many virtual desktops on a small number of physical servers. VMs provide businesses with significant cost savings and scalability to expand capacity as the company, systems, and amount of data grows.
 - **Container:** By comparison, containers run on the host server and its operating system. A runtime system supports the containers. Each container shares the host's system kernel and its shared libraries. Containers are protected from conflicts because the runtime shares components as read-only. If containers need to communicate, they use the network.

- Which is better: VMs or containers? It depends on the need of the organization and its customers. Containers work well for running individual applications, not entire operating systems. Instead of installing an application on the host, a container is run that contains everything the application needs. Containers do not need host resources like a VM does and are lightweight in terms of setup, maintenance, and use as compared to a virtual machine. Containers also have less overhead when compared to servers or VMs. VMs share a common operating system, so maintenance is simplified to patches and bug fixes. Although containers use a variety of different operating systems for their apps, they are self-contained and require less maintenance. However, since only VMs can act like complete systems, in some situations, VMs are the only option.

PI 2.3

Required Practice Information

Practice Statement

Develop, keep updated, and follow procedures and criteria for integrating solutions and components.

Value

Improves the likelihood of producing a solution that works correctly and meets the customer's requirements.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Procedures and criteria for product integration (manual or automated) may address:

- The number of incremental iterations to be performed
- How product components will be verified and validated
- Functionality and quality attributes
- Verification and validation of interfaces or connections
- Thresholds of performance deviation
- The environment to be used for the integration test
- Environmental parameters
- Availability of resources
- Derived requirements for the integration and its external interfaces or connections
- Allowable substitutions of components
- Quality or cost tradeoffs for integration operations
- Delivery rate and variation
- Lead time from order to delivery

Communicate schedule and integration status with affected stakeholders to reduce the risk of delays and failures.

Example Activities

Example Activities	Further Explanation
Develop, use, and keep updated product integration procedures for the product components.	
Develop, use, and keep updated criteria for product component integration and evaluation.	
Record, keep updated, and communicate the product integration procedures and criteria.	

Example Work Products

Example Work Products	Further Explanation
Product integration procedures	
Product integration criteria	

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

Continuous Integration (CI) follows this basic process:

- A developer commits code to the version control repository
- The CI server on the integration build machine periodically polls the source code repository for changes. After a commit, the CI server checks for changes that may have occurred in the version control repository.
- The CI server retrieves the latest version of the code from the repository and then executes a build script, which integrates the software
- The CI server then emails the build results to assigned development team members
- Automated unit tests are then performed if the build succeeds. If the tests are successful, the code is ready to be deployed to either the staging or the production server.
- The CI server continues to poll the source code repository for updates and the process repeats

PI 2.4

Required Practice Information

Practice Statement

Confirm, prior to integration, that each component has been properly identified and operates according to its requirements and design.

Value

Reduces total development cost, integration cycle time, and rework.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Ensure that each component meets its requirements and design and can be integrated according to the product integration strategy and procedures. Components are checked for consistency with interface or connection descriptions. Verification and validation can provide confirmation of integration readiness.

Example Activities

Example Activities	Further Explanation
Track the integration readiness status of the components.	
Ensure components are delivered to the product integration environment as described in the product integration strategy and procedures.	
Confirm each component is correctly identified and received.	
Verify and validate that each received component meets its requirements and design.	
Check the status of the current configuration against the expected configuration.	
Check all the physical interfaces or connections before integrating product components.	For example, check by a visual inspection or using basic counts of interfaces or connections.

Example Activities	Further Explanation
Communicate results with affected stakeholders.	

Example Work Products

Example Work Products	Further Explanation
Acceptance documents or test criteria for each product component	
Exception reports	Include handling instructions for what to do with exceptions and nonconforming work products.

Related Practice Areas

Verification and Validation (VV)

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

Development teams build their integration criteria into their CI/CD pipeline using scripts and a sequence of automated tests. Often, teams use CI tools in a sub-optimal manner by creating environment-specific commands. This approach limits the team to one tool and makes it more difficult to migrate to a different tool set. Ideally, teams write environment agnostic scripts and commands. This minimizes the dependence on a single CI environment and allows the team greater flexibility.

PI 2.5

Required Practice Information

Practice Statement

Evaluate integrated components to ensure conformance to the solution's requirements and design.

Value

Helps to ensure customer requirements are correctly implemented.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

The product integration strategy and procedures define when and how to evaluate solutions and components. For example, each incremental integration can be evaluated to ensure that the components work together. Alternatively, the solution can be evaluated once after all the components are integrated. In either case, the resulting solution should meet its specification which may be defined in requirements, solution architecture, design, or test acceptance criteria.

Evaluate integrated components at different stages of integration as identified in the product integration strategy and procedures. Examine, integrate, and test components for performance, suitability, and readiness using the product integration procedures, criteria, and environment. Verification and validation of the integration testing may be a part of the integration strategy and procedures. Waivers may be used when defects or other results are accepted without resolution.

Example Activities

Example Activities	Further Explanation
Evaluate integrated components, interfaces or connections, and testing using the integration strategy, procedures, and criteria.	
Record and communicate evaluation results.	Example results include: • Integration procedure or criteria changes • Product configuration changes (spare parts, new release) • Deviations from evaluation procedures or criteria • Defects and exceptions

Example Work Products

Example Work Products	Further Explanation
Integration evaluation reports	
Interface or connection evaluation reports	
Test reports	
Exception reports	

Related Practice Areas

Verification and Validation (VV)

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

In a Continuous Integration (CI) environment, a successful build indicates initial verification of the requirements. DevSecOps culture adopts the “shift left” testing approach, which means pushing the testing to the left or into earlier steps of the development process. This culture results in identifying and removing defects early in the lifecycle, improving quality more cost effectively. With this “shift left” approach, testing typically begins when development starts including security testing to identify vulnerabilities and spans the lifecycle, resulting in the integrated solution.

PI 2.6

Required Practice Information

Practice Statement

Integrate solutions and components according to the integration strategy.

Value

Ensures that the customer receives a solution that meets requirements and design.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Integration and evaluation activities can be performed iteratively. Evaluate integrated components before proceeding to the next integration iteration.

Example Activities

Example Activities	Further Explanation
Confirm readiness of the product integration environment.	
Integrate components according to the product integration strategy, procedures, and criteria.	Record all appropriate component information.

Example Activities	Further Explanation
Update the product integration strategy, procedures, and criteria as needed.	
Integrate and deliver the product and communicate results.	Integration and delivery should address requirements for: • Safety • Environment protection • Security • Transportability • Disposal Examples of requirements and standards for packaging and delivering include: • Type of storage and delivery media • Required documentation • Copyrights • License provisions • Preparation of the operational site for product installation Install the product at the operational site and confirm correct operation and communicate results. Final product verification and validation may occur at the operational site.

Example Work Products

Example Work Products	Further Explanation
Integrated solution or components	
Exception or test reports	

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

DevSecOps teams use their Continuous Delivery (CD) pipeline where automated builds, tests, and deployments are orchestrated as one release workflow. Pipelines have software gates that automatically promote or reject versioned components. If the release protocol is not met, the pipeline aborts. Alerts are generated and notifications are sent to the DevSecOps team for remediation.

Level 3

PI 3.1

Required Practice Information

Practice Statement

Review and keep updated interface or connection descriptions for coverage, completeness, and consistency throughout the solution's life.

Value

Reduces rework and missed project objectives caused by incompatible or inconsistent interfaces or connections.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Start managing component interfaces or connections early in the development of the project.

Interfaces or connections include:

- Product component interfaces or connections
- Interfaces or connections with other solutions, systems, or components
- Interfaces or connections with specific environments which may include:
 - Verification
 - Validation
 - Operations
 - Support

Interface or connection definitions and designs can affect verification and validation environments in addition to the components and external systems. Ensure that interface or connection changes are recorded, kept updated, and readily accessible.

Managing interfaces or connections includes maintaining their consistency throughout the life of the solution, complying with architectural decisions and constraints, and resolving conflicts or changes. Managing interfaces or connections between solutions acquired from suppliers and other products or product components is critical for the success of the work.

Example Activities

Example Activities	Further Explanation
Review with affected stakeholders and keep updated interface or connection descriptions for coverage, completeness, and consistency throughout the product's life.	Interfaces or connections are usually classified in three main classes: • Environmental • Physical • Functional Typical categories for these classes may include: • Mechanical • Fluid • Sound • Electrical • Climatic • Electromagnetic • Thermal • Message • Operators or users
Resolve interface or connection issues.	
Update interface or connection descriptions and make accessible to affected stakeholders.	

Example Work Products

Example Work Products	Further Explanation
Interface or connection review results	
List of action items for updating interfaces or connections	
Updated interface or connection descriptions	

Related Practice Areas

Requirements Development and Management (RDM)

Verification and Validation (VV)

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

Understanding system impact is essential to DevSecOps. For example, when applications are built on service-oriented architectures, requirements for one feature may include other system dependencies. Dependency requirements typically take two forms: 1) changes with a potentially negative impact on another part of the application, 2) changes with a dependency on some other part of the application before the feature can be implemented.

Services

Context Tag:	CMMI-SVC
Context:	Use processes to deliver, manage, and improve services to meet customer needs.

In the context of service systems, interfaces or connections can fit into one of four major groups: person-to-person, person-to-component, component-to-component, and compound interfaces:

- Person-to-person interfaces are interfaces or connections that represent direct or indirect communication between two or more people
 - These people can include service provider personnel or end users
 - For example, a call script (which defines how a help desk operator should interact with an end user) defines a direct person-to-person interface
 - Logbooks and instructional signage are examples of indirect person-to-person interfaces or connections
- Person-to-component interfaces are interfaces or connections that encompass interactions between a person and one or more service system components
 - These interfaces or connections can include both graphical user interfaces for automated components, e.g., software applications, and operator control mechanisms for automated, partially automated, and non-automated components, e.g., equipment, vehicles
- Component-to-component interfaces are interfaces or connections that do not include direct human interaction
 - This includes the interfaces of interactions between automated components and other possibilities, such as specifications limiting the physical interaction of two components, e.g., a delivery truck and a loading dock

- Compound interfaces are interfaces or connections that merge or layer together interfaces from more than one of the other three groups
 - For example, an online help system with live chat support might have a compound interface built on an integrated combination of person-to-person, person-to-component, and component-to-component interfaces

Interfaces can be external or internal:

- External interfaces are interactions among components of the service system and any other entity external to the service system, including people, organizations, and systems
- Internal interfaces can include interactions among personnel, teams, and functions of the service provider organization; internal interfaces can also include interaction between personnel or end users and service system components

Examples of user interface work products include:

- Customer interaction scripts
- Reporting types and frequency
- Application program interfaces or connections

PI 3.2

Required Practice Information

Practice Statement

Confirm, prior to integration, that component interfaces or connections comply with interface or connection descriptions.

Value

Reduces the amount of rework due to interface or connection incompatibility.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

This section left blank for future content.

Example Activities

Example Activities	Further Explanation
Compare interface or connection descriptions with component	This comparison may include: • Performing a pre-check of all the physical interfaces or connections before integrating components

Example Activities	Further Explanation
interfaces or connections and identify non-compliances.	- Performing a functional review of external component interfaces or connections - Confirming that verification and validation activities were completed
Address interface or connection non-compliances and communicate results.	

Example Work Products

Example Work Products	Further Explanation
Results of comparison of component interfaces or connections to their descriptions	
List of component interface or connection non-compliances	
List of action items for updating interface or connection descriptions or component interfaces or connections	
Updated interface or connection descriptions or component interfaces or connections	

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

As the code moves through the Continuous Integration (CI) process, a series of automated unit tests are conducted. When using a CI server, eventually a build fails on the CI due to differences in the environment but passes on the development machine. When this happens, the configuration issues, language version, package installation, memory limitations, test user permissions, etc., from the passed tests are eliminated systematically and the issue is narrowed down to what has changed recently with the build, and any unmet integration requirement is identified and resolved.

Services

Context Tag:	CMMI-SVC
Context:	Use processes to deliver, manage, and improve services to meet customer needs.

Integrate the service system as defined in the planned integration strategy and procedures. Before integration, verify each service system component for compliance with its interface or connection requirements, including any service system components that are manual processes.

PI 3.3

Required Practice Information

Practice Statement

Evaluate integrated components for interface or connection compatibility.

Value

Reduces the risk of interface or connection failure within integrated components.

Additional Required Information

This section left blank for future content.

Explanatory Practice Information

Additional Explanatory Information

Evaluate and test integrated components to ensure that the interfaces or connections within components are functioning correctly.

As identified in the product integration strategy and procedures, evaluate integrated components and their interfaces or connections, as appropriate, at different stages of integration.

Example Activities

Example Activities	Further Explanation
Evaluate the integrated components for compatibility.	Compatibility may include: • Alignment with specifications • Functionality • Reliability
Record and communicate the evaluation results.	Example results include: • Integration procedure or criteria changes • Product configuration changes (spare parts, new release) • Interface or connection and interface or connection description changes • Deviations from evaluation procedures or criteria • Interface or connection defects

Example Work Products

Example Work Products	Further Explanation
Interface or connection issues reports	

Context Specific

DevSecOps

Context Tag:	DevSecOps
Context:	DevSecOps is a mindset, a culture, and set practices that fosters close cooperation between development, operations, and security to plan, develop, test, deploy, release, and keep updated a secure solution.

CI requires automated unit and integration tests. These tests must be comprehensive enough so that teams have confidence that the software works as expected, vulnerabilities are identified and fixed, and meets the requirements. The unit tests must run quickly so that they do not delay development when check-in occurs. Complex automated tests are typically run overnight so that developers have the results the next day and can be discussed. The tests must be run frequently and be kept updated. If the tests are run infrequently, then a test failure can originate from many different changes, making it much harder and more expensive to debug and more complicated to keep updated.

Creating maintainable sets of automated unit tests is difficult and complex. An effective way to solve this problem is to practice Test-Driven Development (TDD), where developers write automated tests that initially fail, until they implement code that makes the tests pass. TDD helps ensure developers write code that is modular and easy to test, which reduces the maintenance of the resulting automated test suites.

Services

Context Tag:	CMMI-SVC
Context:	Use processes to deliver, manage, and improve services to meet customer needs.

Some service systems can require assembly with customer or end user resources to complete full integration. When these resources are available under the terms of a service agreement, incorporate them, as appropriate, in integration activities. When such resources are not available from customers and end users, temporarily employ substitute equivalent resources to enable full-service system integration.